MEMORIA FINAL

MadriDeep AI

Tasación Inteligente del Mercado Inmobiliario

Bootcamp KeepCoding

Big Data 16

Autores:

Alfredo Naranjo Serres
Alvar Garcia Moral
Jhan Franco Schotborgh Piersanti
María del Rocío Sánchez Quintana

ÍNDICE

INTRODUCCIÓN

1. ORIGEN Y ADQUISICIÓN DE DATOS
1.1. Fuentes de Datos y Procedencia
1.2. Naturaleza y Tipología de las Variables
1.3. Diccionario de Datos del Proyecto

2. ARQUITECTURA DEL SISTEMA Y PIPELINE TECNOLÓGICO
2.1. Diagrama de Flujo y Ecosistema de Datos
2.2. Descripción Detallada de las Fases del Pipeline

3. ANÁLISIS EXPLORATORIO DE DATOS (EDA) Y PREPROCESAMIENTO
3.1. Limpieza de Datos y Gestión de Outliers
3.2. Análisis de Correlaciones
3.3. Segmentación del Mercado y Distribución
3.4. Transformación y Preparación Final

4. MODELADO: LA BATALLA POR LA PRECISIÓN
4.1. Machine Learning Clásico (El Campeón)
4.1.1. Breve explicación de los algoritmos utilizados
4.2.1. Planteamiento y entrenamiento
4.2.2. Separación de features y variable objeto
4.2.3. Entrenamiento de los modelos con hiperparámetros por defecto (los ‘baseline’)
4.2.4. Estrategia de Optimización (fine tuning)
4.2.5. Evaluación y conclusiones en ML
4.2.6. Puntos a mejorar para siguientes iteraciones

5. INFRAESTRUCTURA Y MLOPS: EL CAMINO A PRODUCCIÓN
5.1. Organización y Estructura del Proyecto
5.2. Control de Versiones y Desarrollo Iterativo
5.3. El Selector Inteligente de Modelos (Automatización)
5.4. Interfaz de Usuario e Integración de LLM
5.4.1. Visualización Avanzada (Power BI)
5.5. Entorno de Ejecución y Dependencias

6. PRODUCTO FINAL
6.1. Demostración del Producto: MadriDeep AI

7. IMPACTO DE NEGOCIO Y PROPUESTA DE VALOR

7.1. Problema de Negocio
7.2. Propuesta de Valor
7.3. Impacto Económico Potencial
7.4. Modelo de Negocio

8. CONCLUSIONES
8.1. Conclusiones Técnicas
8.2. Consecución de Objetivos
8.3. Futuras Líneas de Mejora

INTRODUCCIÓN

En la actualidad, el mercado inmobiliario en la Comunidad de Madrid presenta una complejidad sin precedentes. La alta demanda, sumada a la volatilidad de los precios tanto en venta como en alquiler, hace que la toma de decisiones para compradores e inquilinos sea una tarea ardua y, a menudo, carente de datos objetivos.

MadriDeep AI nace como una solución tecnológica avanzada diseñada para democratizar el acceso a la inteligencia de datos en el sector inmobiliario. Este proyecto consiste en el desarrollo de un Asistente de Inteligencia Artificial capaz de tasar viviendas con cierta aproximación de precisión y ofrecer asesoramiento personalizado mediante lenguaje natural.

A través de la combinación de algoritmos de Machine Learning, redes neuronales de Deep Learning y modelos de lenguaje de última generación, MadriDeep AI permite a los usuarios:

- Obtener valoraciones precisas basadas en características reales y actualizadas.
- Comprender las tendencias específicas de cada uno de los 21 distritos madrileños.
- Interactuar con un consultor virtual disponible 24/7 para resolver dudas complejas sobre inversión, financiación o habitabilidad.

Este documento detalla el ciclo de vida completo del proyecto, desde la ingesta y curación de datos brutos hasta el despliegue de la aplicación web final.

1. ORIGEN Y ADQUISICIÓN DE DATOS

El éxito de cualquier modelo predictivo reside en la representatividad y veracidad de sus datos. En esta fase inicial, el objetivo fue construir un dataset robusto que capturara la esencia del mercado madrileño. El archivo resultante, madrid_real_estate_market.csv, es el fruto de una integración estratégica de varios pilares de información.

1.1. Fuentes de Datos y Procedencia

Para garantizar un entrenamiento libre de sesgos y alineado con la realidad económica, se ha aplicado una metodología de triangulación de fuentes:

1. Dataset de Soporte (Kaggle): Se ha integrado una base de datos histórica con más de 21.740 registros. Este volumen masivo permite que los algoritmos identifiquen patrones estadísticos sólidos y comportamientos históricos del mercado.

Referencia: Mercado inmobiliario de Madrid en Kaggle.

2. Obtención de data de muestra procedente de portales inmobilirarios: Se ha utilizado información extraída de anuncios activos para aportar "granularidad" técnica: estado de conservación, orientación, altura del piso y extras (ascensor, piscina, etc.).

Referencia: Fotocasa.es — Mercado Madrid e Idealista, por ejemplos.

1.2. Naturaleza y Tipología de las Variables

El dataset consolidado se divide en tres dimensiones principales:

- Variables Espaciales: pertenencia a distritos.
- Variables Estructurales: Metros cuadrados, número de dormitorios, aseos y año de construcción, entre otras variable de interés y amenities.
- Variables de Confort: Equipamiento, eficiencia energética y servicios comunes.

1.3. Diccionario de Datos del Proyecto

A continuación, se describen las variables fundamentales para el entrenamiento de los modelos de predicción:

Variable	Descripción	Tipo
subtitle	Nombre del barrio	Categórico
sq_mt_built	Superficie total construida (m²).	Numérico
n_rooms	Número de dormitorios.	Numérico
n_bathrooms	Número de cuartos de baño completos.	Numérico

floor	Número del piso al que pertenece	Númerico - Ordinal
is_floor_under	Indica si la propiedad es subterránea	Booleano
house_type_id	Indica el tipo de propiedad	Categórico
is_renewal_needed	Indicador de necesidad de renovación de la propiedad	Booleano
is_new_development	Indicador de propiedad nueva	Booleano
is_exterior	Indica si la vivienda es exterior.	Booleano
has_lift	Disponibilidad de ascensor en la finca.	Booleano
energy_certificate	Indicador de categorías de eficiencia energética	Categórico - Ordinal
has_parking	Indicador de existencia de parking en la propiedad	Booleano
Distrito	Distrito donde se ubica la propiedad	Categórico
buy_price	**Precio de venta total de la propiedad. (Target)**	**Numérico**
rent_price	**Precio mensual del alquiler. (Target)**	**Numérico**

2. ARQUITECTURA DEL SISTEMA Y PIPELINE TECNOLÓGICO

Para garantizar la escalabilidad y eficiencia de MadriDeep AI, se ha diseñado una arquitectura modular basada en un "Pipeline" de datos extremo a extremo (End-to-End). Este diseño permite que el flujo de información transite de forma fluida desde su estado bruto hasta la interacción final con el usuario, asegurando la trazabilidad de cada proceso.

2.1. Diagrama de Flujo y Ecosistema de Datos

A continuación, se representa el ecosistema tecnológico real del proyecto. El diagrama detalla la interconexión entre las fuentes de datos, los notebooks de entrenamiento y la lógica de selección automática de modelos alojada en el repositorio:

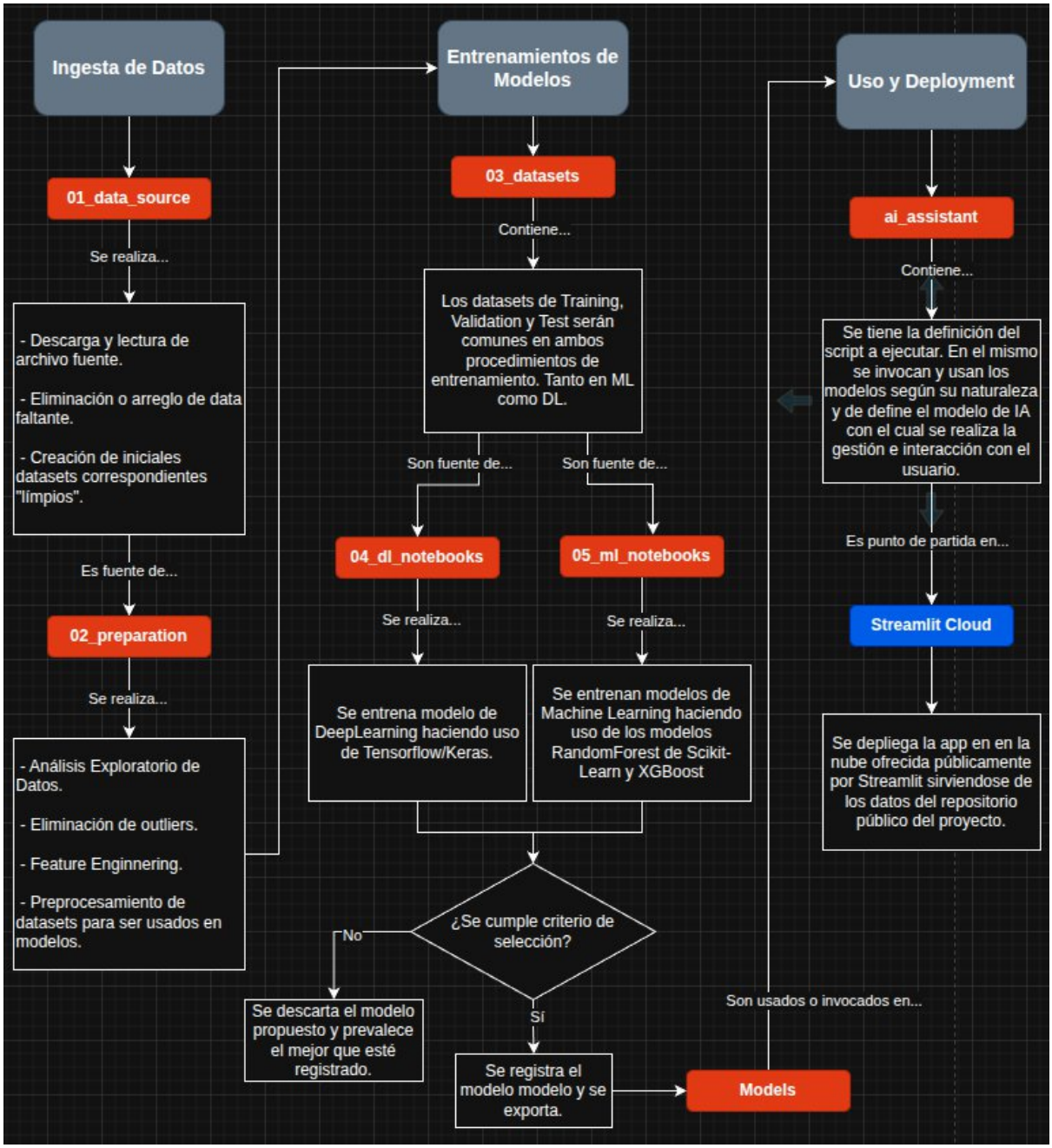


Figura 1. Diagrama de flujo y ecosistema de datos del pipeline End-to-End de MadriDeep AI.

Como se observa, el flujo de datos comienza con una fase de Ingesta y Preparación que unifica el origen de los datos CSV y su posterior limpieza antes de alimentarlos a la Capa de Inteligencia Predictiva. Luego se procede al aplicar el criterio de registro y selección de modelo que será usado por el asistente para la interacción con el usuario en la nube de Streamlit

2.2. Descripción Detallada de las Fases del Pipeline

2.2.1. 01_data_source y 02_preparation: Ingesta y Preparación

Esta fase unifica el origen y la calidad del dato antes de cualquier modelado, asegurando que la información sea robusta:

- 01_data_source: Consolidación de archivos CSV provenientes de tres pilares: Kaggle (histórico) y la obtención de datos portales de internet (actualidad del mercado).
- 02_preparation: Uso de Python (Pandas/Numpy) para la gestión de nulos, eliminación de outliers y normalización de variables. El resultado final se almacena en la carpeta 03_datasets, que contiene los conjuntos listos para entrenamiento y validación.

2.2.2. 04_dl y 05_ml: Capa de Inteligencia y Modelado

El núcleo predictivo utiliza una arquitectura híbrida donde compiten dos enfoques tecnológicos para garantizar la mejor tasa de acierto:

- 05_ml_notebooks (Machine Learning): Implementación de algoritmos de XGBoost y Random Forest mediante Scikit-Learn.
- 04_dl_notebooks (Deep Learning): Desarrollo de Redes Neuronales multicapa utilizando la librería TensorFlow/Keras.

2.2.3. models: Registro y Selección Automática (Punto 5)

En lugar de depender de herramientas externas, el proyecto implementa un sistema de MLOps personalizado dentro de la carpeta models. La lógica de entrenamiento consulta y actualiza el archivo models_log.csv para comparar el rendimiento mediante la métrica R^2:

- Criterio de Selección: Solo se exporta y activa el modelo (archivo .joblib o .keras) que supere la precisión del mejor modelo registrado anteriormente.
- Consistencia: Se asegura que el archivo preprocessor.joblib coincida siempre con el modelo activo para garantizar la estabilidad de las predicciones en tiempo real.

2.2.4. ai_assistant y app.py: Despliegue y Frontend (Punto 6)

La interfaz final se materializa mediante Streamlit (app.py). Este Frontend integra las capacidades de la carpeta ai_assistant, donde reside la lógica conversacional:

- Extractor de Entidades (LLM): Utiliza el modelo Llama-3.3-70b (vía Groq) para analizar el texto libre de un anuncio inmobiliario y extraer automáticamente los campos estructurados que el modelo predictivo necesita (metros, habitaciones, distrito, extras)
- Asesor Conversacional (LLM)**:** El mismo modelo Llama-3.3-70b gestiona un chatbot con memoria de contexto real, acumulando el historial de la conversación en cada llamada. Actúa como experto inmobiliario exclusivo de Madrid con guardrails estrictos que limitan las respuestas al dominio del proyecto.
- Tasación Instantánea: El asistente invoca los modelos ganadores de la fase anterior para devolver una valoración inmobiliaria precisa de forma inmediata.

3.ANÁLISIS EXPLORATORIO DE DATOS (EDA) Y PREPROCESAMIENTO

Tras la fase de ingesta de datos, se procedió a una etapa crítica de análisis exploratorio. El objetivo es comprender las dinámicas del mercado inmobiliario madrileño y asegurar que los datos estén optimizados para los algoritmos de aprendizaje automático.

Para garantizar la transparencia en el tratamiento de la información, se define el siguiente Diccionario de Variables que constituye el núcleo de MadriDeep AI:

Variable	Tipo	Descripción
sq_mt_built	Numérica	Superficie total construida en metros cuadrados.
n_rooms	Numérica	Número de habitaciones declaradas.
n_bathrooms	Numérica	Número de cuartos de baño completos.
buy_price	Numérica	Precio de venta final (Variable objetivo).
is_renewal_needed	Booleana	Indica si la vivienda requiere reforma (1) o no (0).
has_lift	Booleana	Presencia de ascensor en la finca (1/0).
is_exterior	Booleana	Orientación exterior (1) o interior (0).
Distrito	Categórica	Ubicación geográfica (21 distritos de Madrid).

3.1. Limpieza de Datos y Gestión de Outliers

La fiabilidad del modelo depende de la calidad de la muestra. Se realizaron limpiezas de registros con valores nulos e inconsistencias. Para la gestión de valores atípicos, se utilizaron diagramas de caja.

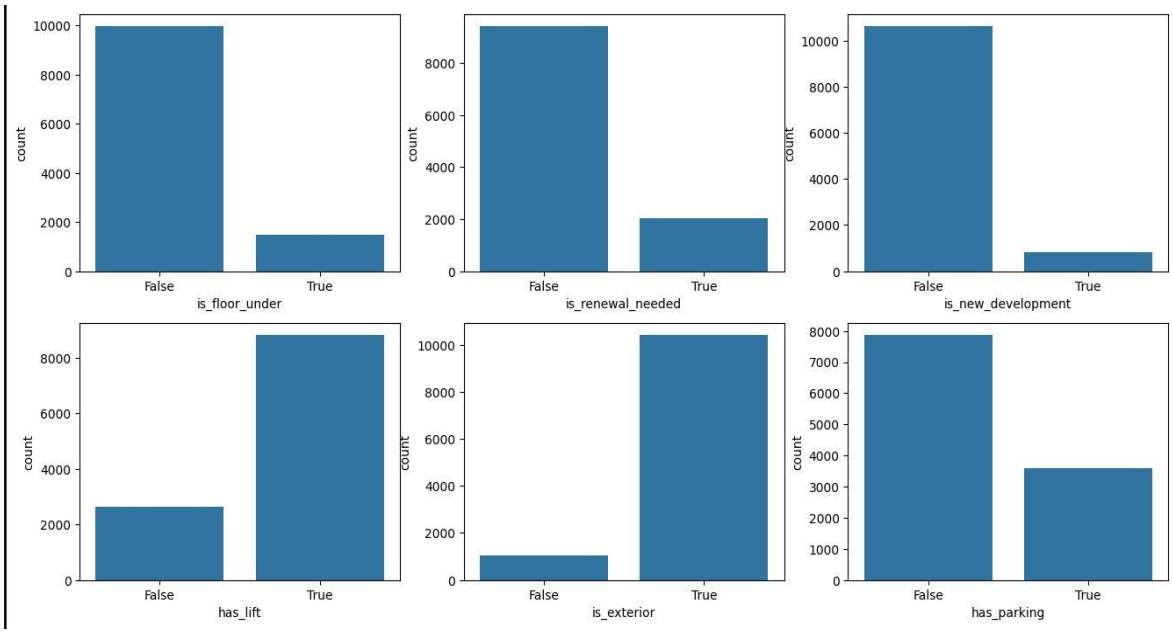


Figura 2. Análisis de Outliers en el Precio de Compra (buy_price): distribución original y transformada logarítmica.

Como se observa en los Boxplots, el mercado presenta una alta dispersión. Se aplicaron filtros estadísticos para eliminar propiedades con precios de superlujo o metros cuadrados irreales que habrían introducido ruido en el entrenamiento del modelo generalista.

3.2. Análisis de Correlaciones

Para determinar qué factores influyen más en la tasación, se analizó la distribución de las variables numéricas discretas clave: el número de baños (n_bathrooms) y el número de habitaciones (n_rooms).

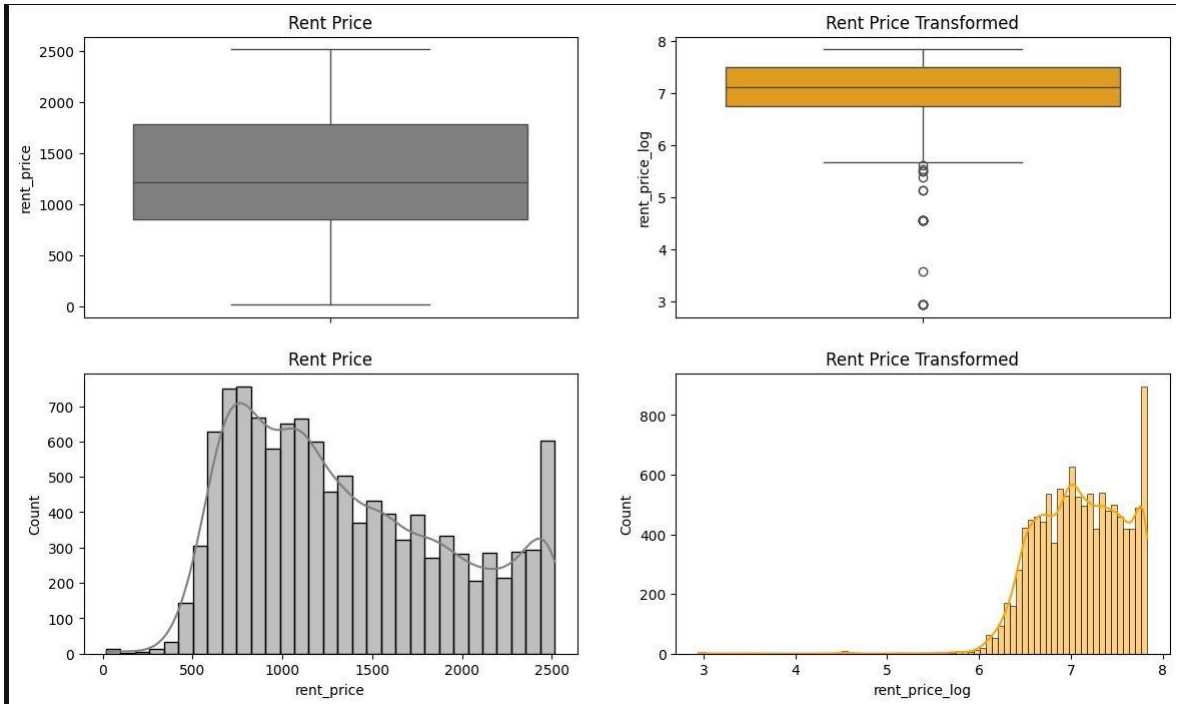


Figura 3. Distribución y valores atípicos de n_bathrooms y n_rooms.

Hallazgos clave:

- Existe una correlación muy fuerte (~0.82) entre los metros cuadrados y el precio.
- El número de baños presenta una correlación superior al número de habitaciones, indicando que el mercado valora más la densidad de servicios que el número de estancias.

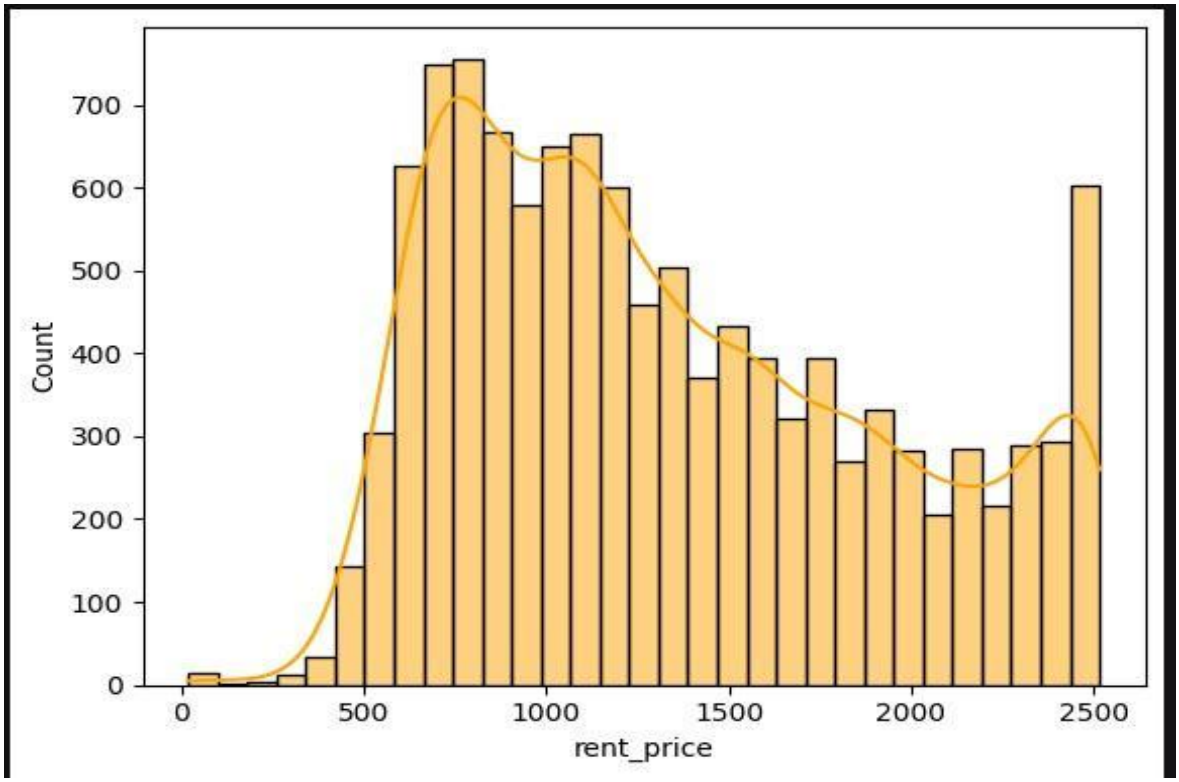


Figura 4. Distribución del precio de alquiler (rent_price) con curva KDE.

El histograma del precio de alquiler muestra una distribución bimodal con un pico principal en torno a 800€ y un segundo repunte cerca de 2.500€, lo que justifica la aplicación de una transformación logarítmica para normalizar la variable antes del modelado.

3.3. Segmentación del Mercado y Distribución

Se analizaron las características cualitativas de las viviendas mediante variables booleanas, y la distribución del precio de alquiler antes y después de la transformación logarítmica.

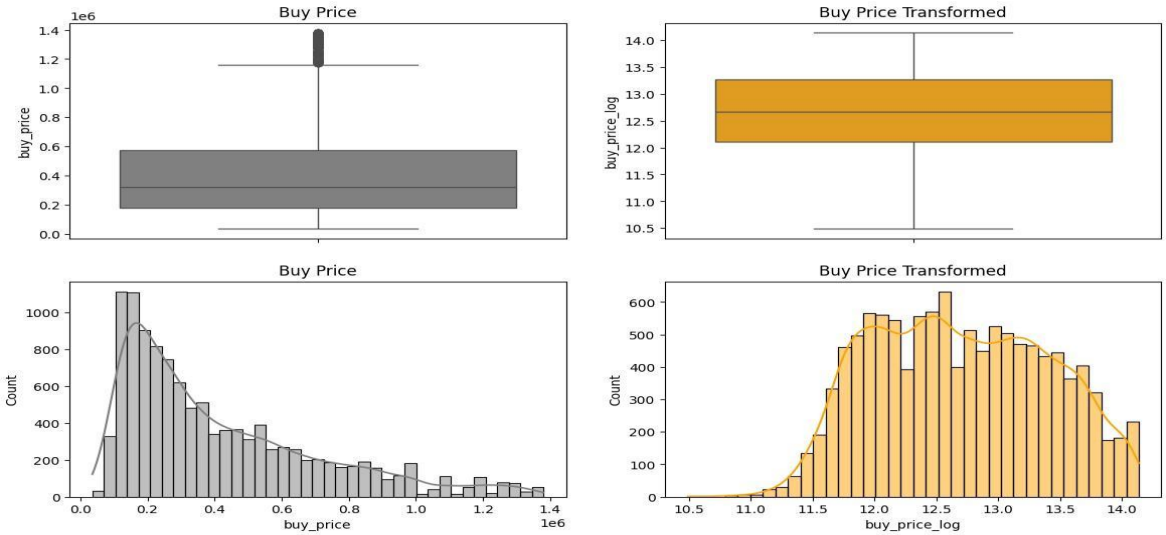


Figura 5. Distribución de variables booleanas: ascensor, exterior, parking, reforma, obra nueva y bajo.

La mayoría de las viviendas cuentan con ascensor (has_lift) y son exteriores (is_exterior), mientras que el parking y la obra nueva son menos frecuentes. Estas características permiten al modelo capturar el perfil de confort de cada inmueble y su impacto en el precio.

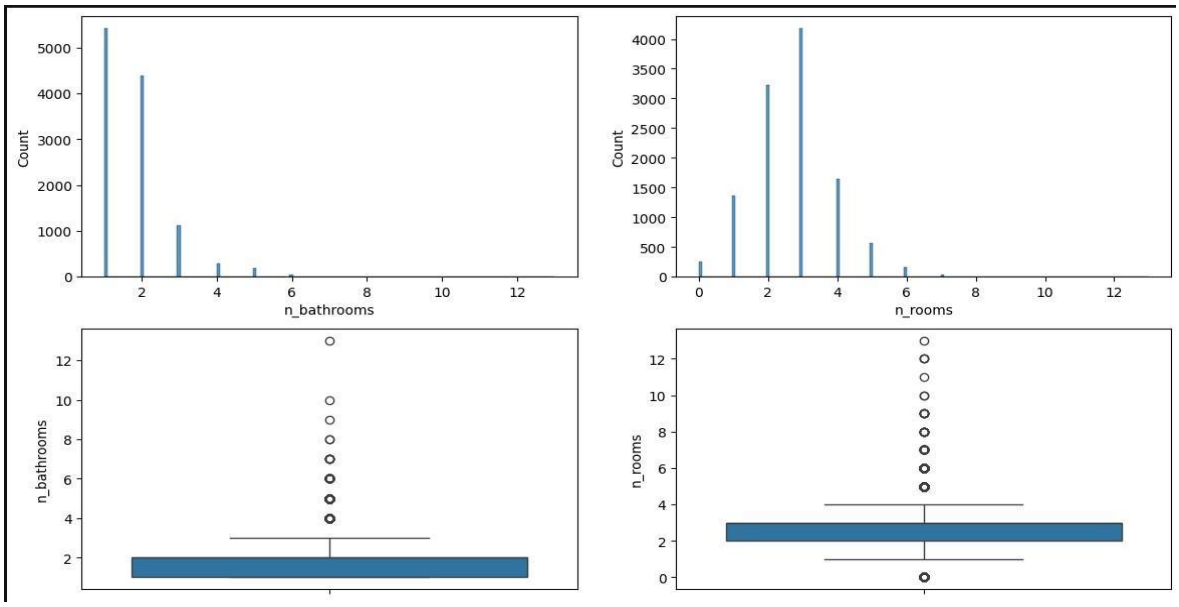


Figura 6. Precio de alquiler (rent_price): distribución original y transformada logarítmica.

La transformación logarítmica del precio de alquiler mejora notablemente la simetría de la distribución, reduciendo el impacto de los valores extremos y facilitando la convergencia de los algoritmos de aprendizaje automático.

3.4. Transformación y Preparación Final

Para optimizar el rendimiento de la Red Neuronal y el Random Forest, se aplicaron transformaciones y procesos de ingeniería de características (Feature Engineering):

1. Normalización Logarítmica: Se transformó la variable buy_price a buy_price_log. Esto reduce el sesgo de los valores extremos y genera una distribución gaussiana (forma de campana), facilitando la convergencia del algoritmo.
2. Las variables: **subtitle** y **Distrito** fueron transformadas usando el método de **Target Encoding**.
3. La variable: **house_type_id** fue sometida al método **One-Hot Encoding**.
4. La variable: **energy_certificate** fue transformada usando método **Ordinal Encoding**.
5. Las variables: **is_floor_under**, **is_renewal_needed**, **is_new_development**, **has_lift**, **is_exterior**, **has_parking** se transformaron usando **One-Hot Encoding**.
6. Las variables numéricas: **sq_mt_built**, **n_rooms**, **n_bathrooms**, **floor** se les aplicó **StandardScaler**.
7. El Target: **buy_price** ha sidos transformado a una escala logaritmica de log1p.
8. El Target: **rent_price** se mantendrá en su escala original.
9. La variable **buy_price_by_area** se elimina porque suministra información que representa data leakage.

10. La variable **Precio_por_m2** elimina porque tanto distrito como subtitle podrían suministrar la información similar a ésta.

4. MODELADO: LA BATALLA POR LA PRECISIÓN

En esta fase, el objetivo principal fue desarrollar un motor de tasación capaz de entender la complejidad del mercado madrileño. La estrategia se basó en una "batalla de modelos": comparamos algoritmos de Machine Learning (ML) Clásico contra Redes Neuronales de Deep Learning (DL) para garantizar la mayor estabilidad.

4.1. Machine Learning Clásico

Se entrenaron 2 modelos: de Random Forest y XGBoost. Estos algoritmos basados en árboles demostraron una capacidad excelente para capturar las relaciones no lineales del mercado inmobiliario (barrios, metros, extras), arrojando métricas de R-cuadrado (R2) y MAE altamente competitivas.

Cabe resaltar que se elaboraron dos notebooks: uno para predecir la variable de precio de compra y otro para predecir la variable de precio de alquiler. Con el fin de no extenderse más de lo necesario, se explicará la elaboración de un notebook, el que corresponde al entrenamiento de modelos para la predicción del precio de compra.

4.1.1. Breve explicación de los algoritmos utilizados:

El modelo a elaborar planteaba el entrenamiento de dos algoritmos de aprendizaje supervisado para un problema de regresión usando distintos métodos para posteriormente elegir el que mejor resultados arrojase. A continuación se explican estos dos modelos:

Random Forest (Bosque Aleatorio):

Es un método de bagging (Bootstrap Aggregating).

Cómo funciona: Construye cientos o miles de árboles de decisión de forma independiente y en paralelo. Cada árbol se entrena con una submuestra aleatoria de los datos (bootstrap) y utilizando un subconjunto aleatorio de características (features).

- Predicción: Para regresión, el resultado final se obtiene mediante el promedio de todas las predicciones.

Este modelo es robusto frente al sobreajuste (overfitting), y maneja bien datos faltantes y ruidosos, y requiere menos ajuste de hiperparámetros. En el caso de nuestro dataset, el problema de los datos faltantes se elimina debido al tratamiento previo de preparación realizado sobre el dataset.

XGBoost (eXtreme Gradient Boosting):

XGBoost es una implementación optimizada del algoritmo de *Gradient Boosting* diseñada para ser altamente eficiente, flexible y portátil. Pertenece a la familia de algoritmos de **ensamble**, construyendo modelos de forma secuencial donde cada nuevo árbol de decisión intenta corregir los errores residuales de los anteriores.
Su éxito en entornos productivos se debe a tres pilares:

- **Regularización Formal:** Incorpora penalizaciones L1 (Lasso) y L2 (Ridge) en su función objetivo, lo que controla la complejidad del modelo y previene el sobreajuste (*overfitting*).
- **Manejo de Datos Dispersos:** Posee un algoritmo de búsqueda de divisiones (*splits*) consciente de la escasez, optimizando el tratamiento de valores nulos o *one-hot encoding*.
- **Eficiencia Computacional:** Implementa estructuras de bloques para el procesamiento en paralelo y el uso de caché, permitiendo entrenar sobre grandes volúmenes de datos en fracciones del tiempo que requieren otros algoritmos de *boosting*.

Fine-tuning en XGBoost:

El Fine-tuning es el proceso de ajuste de los hiperparámetros del modelo para maximizar su capacidad de generalización sobre datos no vistos. En XGBoost, este proceso es crítico debido a la alta sensibilidad del algoritmo a parámetros como la profundidad del árbol (max_depth), la tasa de aprendizaje (learning_rate) y los términos de regularización (gamma, alpha, lambda). Un ajuste correcto permite encontrar el punto óptimo en el compromiso entre sesgo y varianza (bias-variance tradeoff).

4.2.1. Planteamiento y Entrenamiento:

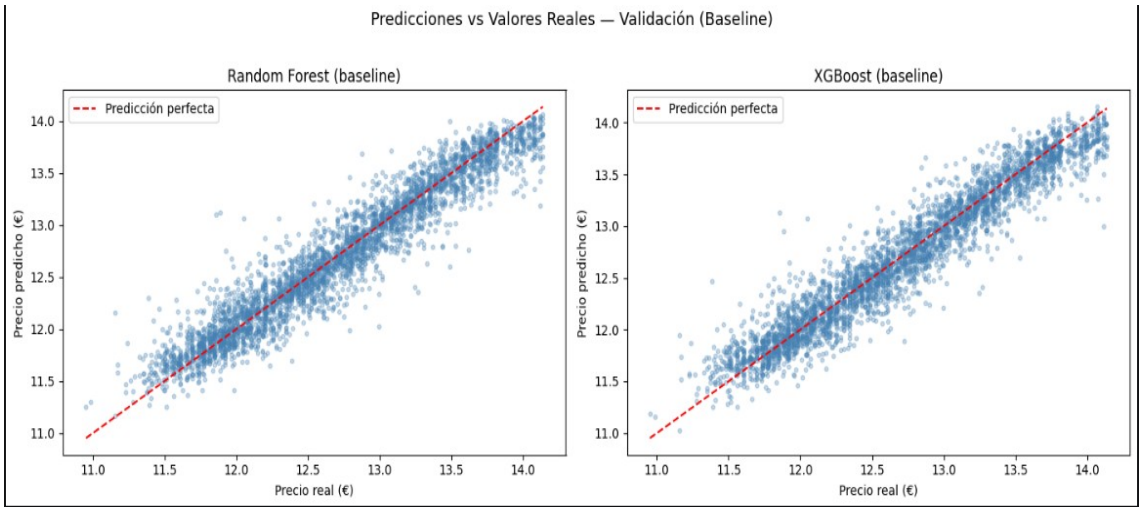
La elaboracion de los modelos empezó con la instalación e importación de librerias a usar, como también de los tres splits del dataset previamente tratados y preparados para el entrenamiento.

4..2.2. Separación de features y variable objeto::

Para la elaboración de los modelos, es clave separar la variable ‘target’ (en el primero de los notebooks que se elaboraron, la variable ‘target’ fue el precio de compra ‘buy_price’ (convertido previamente a escala logarítmica ‘buy_price_log’). Cabe resaltar que se deja fuera del dataframe de variables predictoras el precio de alquiler (‘rent_price’) para evitar data leakage. El resto de variables que se usan para predecir dicha variable. Para esto se hace una partición del dataset donde las variables ‘predictoras’ quedan en un dataframe X (que contenía 16 variables predictoras) y el ‘target’ en un dataframe Y (con una única variable a predecir).

4.2.3. Entrenamiento de los modelos con hiperparametros por defectos (o ‘baseline’):

El siguiente paso en la elaboración del notebook era el paso de definición de los modelos ‘baseline’, o dicho de otra forma, con hiperparámetros por defecto.



Para comparar los modelos ‘baseline’ se lleva a cabo una evaluación del conjunto de validación usando métricas de regresión:

- **MAE** (Error Absoluto Medio): error promedio en euros.
- **RMSE** (Raíz del Error Cuadrático Medio): penaliza más los errores grandes.
- **R^2** (Coeficiente de determinación): proporción de varianza explicada (1.0 = perfecto)

los resultados de esta comparación fueron los siguientes:

```
📊 Resultados en VALIDACIÓN (baseline):

=== Random Forest (baseline) ===
  MAE:  60,537.6744 €
  RMSE: 98,568.9737 €
  R²:   0.893023

=== XGBoost (baseline) ===
  MAE:  61,506.7735 €
  RMSE: 97,864.7819 €
  R²:   0.894546
```

Los valores de error absoluto (MAE) en ambos modelos se encuentran próximos a 60.000 euros. En el caso de pisos que valgan al rededor de 300.000 euros esto supone un 20%, que es un valor de error algo alto pero aceptable, pero en el caso de los pisos con valores próximos a los 150.000 euros (algo escasos en Madrid en los últimos años) supone un error del 40%, que se torna poco aceptable. El modelo con el mejor valor en esta métrica fue RandomForest, significando que es más robusto en cuanto al error medio.

Los valores de la Raíz del Error Cuadrático Medio (RMSE) son altos para ambos modelos, señalando a un elevado nivel de error a la hora de predecir outliers. Cabe destacar que el modelo de XGBoost es ligeramente mejor prediciendo estos casos.

La capacidad explicativa (R^2) de ambos modelos es alta, con ambos modelos capaz de explicar más del 89% de la muestra, siendo el XGBoost ligeramente mejor.

4.2.4. Estrategia de Optimización (Fine-tuning)

Para la elección de los hiperparámetros de los modelos, utilizamos RandomizedSearchCV para explorar miles de combinaciones de hiperparámetros y encontrar el punto óptimo entre sesgo y varianza. Tras este afinado, se volvió a evaluar las metricas de regresión, cuyos resultados se expresan a continuación:

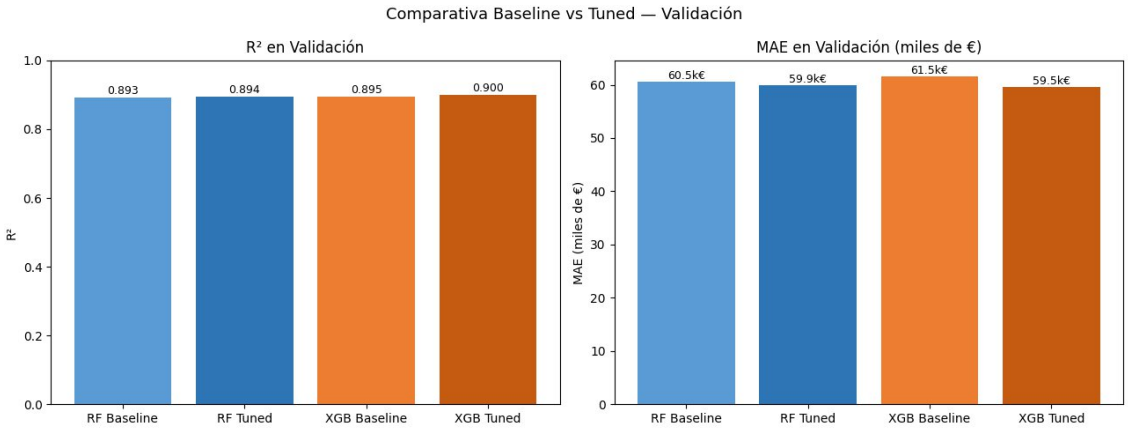
```
📊 Resultados en VALIDACIÓN (tras tuning):

=== Random Forest (tuned) ===
  MAE:  59,874.5922 €
  RMSE: 98,206.4173 €
  R²:   0.893809

=== XGBoost (tuned) ===
  MAE:  59,538.5975 €
  RMSE: 95,457.0022 €
  R²:   0.899672
```

Como se puede observar, tanto los valores de MAE, RMSE y R^2 han mejorado para ambos modelos, aunque la mejora es más pronunciada en el modelo XGBoost.

A continuación se muestra una tabla comparando R2 y MAE de los modelos ‘baseline’ contra los ‘tuned’:



4.2.5. Evaluación de Conclusiones en ML:

A continuación se muestra la comparación de las métricas de ambos modelos en los splits de validación y test:

📋 TABLA RESUMEN FINAL

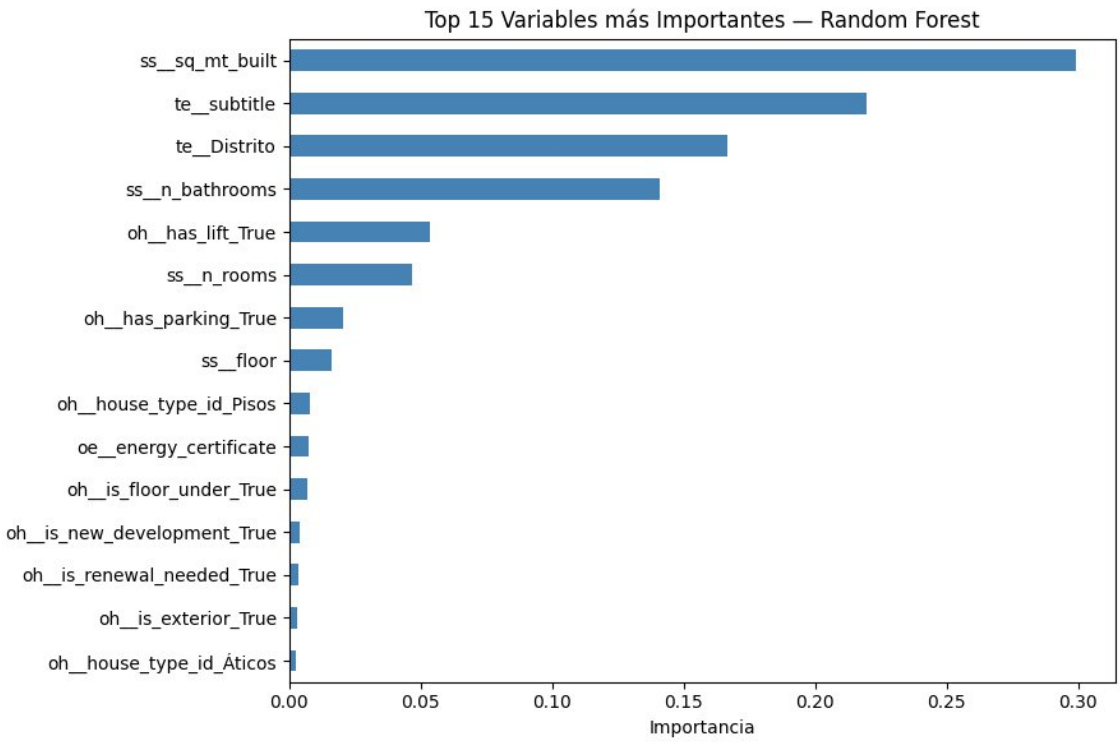
Modelo	MAE Val (€)	RMSE Val (€)	R^2 Val	MAE Test (€)	RMSE Test (€)	R^2 Test
Random Forest	59875.0	98206.0	0.8938	61606.0	96699.0	0.8966
XGBoost	59539.0	95457.0	0.8997	60185.0	93409.0	0.9035

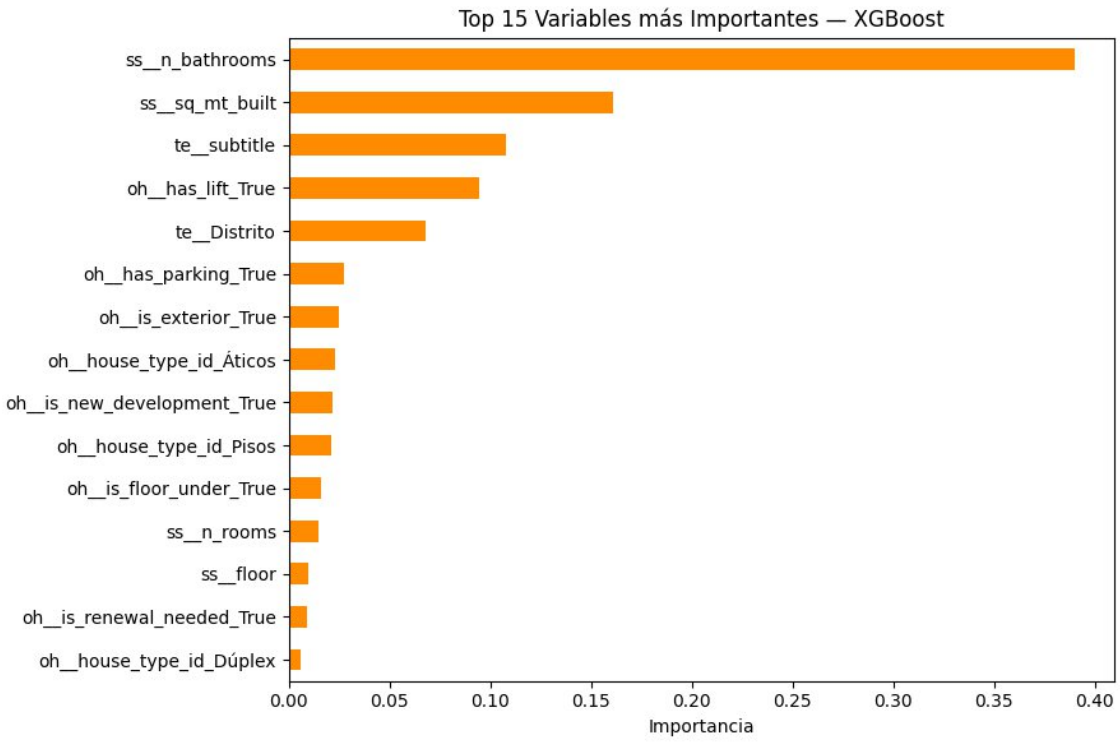
Como se puede observar, aunque el MAE en test es ligeramente mayor, tanto el RMSE y el R^2 es menor en el split de test que en el split de validación. Esto indica que el modelo no presenta leakage ni overfitting aparente, y ambos modelos consiguen explicar bastante bien el dataset, aunque el modelo XGBoost es claramente superior, rompiendo la barrera del 90% del dataset explicado. Sigue presentando un RMSE más alto de lo que sería óptimo.

Para remediar esto, se recomienda, en próximas iteraciones, un tratado adicional de los outliers, la transformación de la variable target, y quizás también un corte más estricto en la elección de features usadas en el entrenamiento de los modelos.

4.2.6. Puntos a mejorar para siguientes iteraciones:

Para remediar esto, se recomienda, en próximas iteraciones, un tratado adicional de los outliers, la transformación de la variable target, y quizás también un corte más estricto en la elección de features usadas en el entrenamiento de los modelos. A este efecto, se presentan a continuación dos gráficas donde se observa la importancia de cada una de las variables usadas en el entrenamiento de los modelos. Como se puede ver a simple vista, existen un número de variables cuya importancia es ínfima. Se recomienda para próximas iteraciones la eliminación de todas las variables cuya importancia tenga un valor por debajo del presentado por la variable ‘ss_floor’.





4.2. El Desafío del Deep Learning

Se implementaron Redes Neuronales Densas con Keras. Sin embargo, se encontró una gran inestabilidad matemática al trabajar con los precios de compra.

- El Problema: Al predecir el logaritmo del precio, pequeñas desviaciones generaban errores masivos en la tasación final (valores astronómicos).
- La Solución: Se refactorizó la red incluyendo capas de BatchNormalization para normalizar las activaciones y evitar el gradiente explosivo.

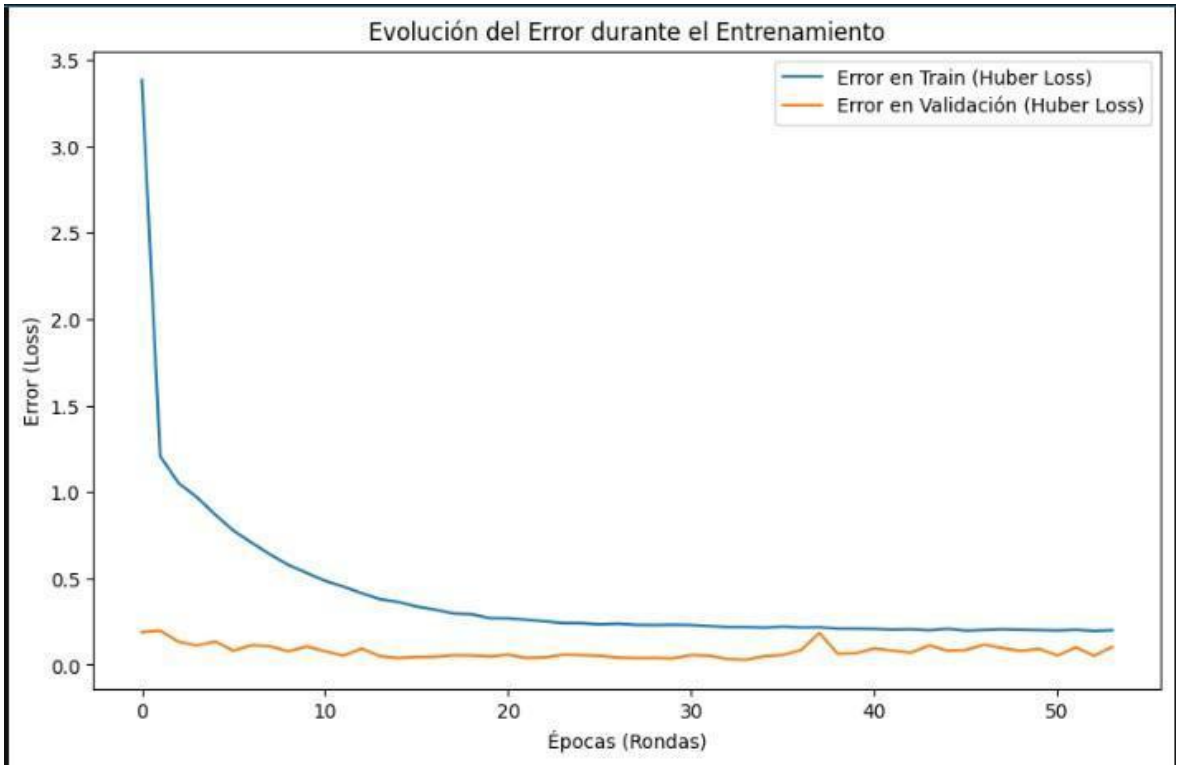


Figura 11. Arquitectura de la Red Neuronal para Compra con capas de normalización (BatchNormalization).

4.3. Conclusión y Selección del Modelo

Comparando los resultados registrados en el historial, el Random Forest se consolidó como el motor de producción debido a su robustez frente a valores atípicos.

Mercado	Modelo Ganador	Precisión (R^2)	Error Medio (MAE)
Compra	Random Forest	92,4%	59.896 €
Alquiler	Random Forest	82,6%	149,85 €/mes

4.4. MLOps y Persistencia

Para profesionalizar el ciclo de vida, se implementó un sistema que guarda automáticamente el mejor modelo.

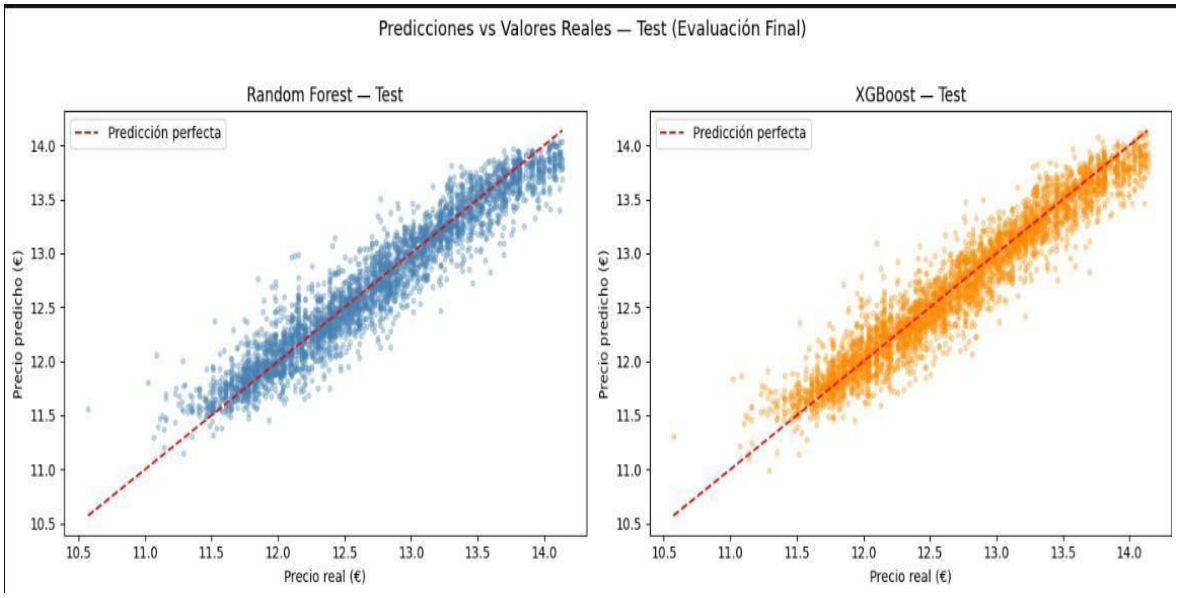


Figura 12. Lógica de guardado y persistencia del modelo "Campeón".

5. INFRAESTRUCTURA Y MLOPS: EL CAMINO A PRODUCCIÓN

No basta con tener un modelo con buena precisión; el valor real de MadriDeep AI reside en su capacidad para ser utilizado de forma sencilla y fiable por cualquier usuario. En este capítulo se detalla la infraestructura técnica y las prácticas de gestión del ciclo de vida del modelo (MLOps) implementadas.

5.1. Organización y Estructura del Proyecto

Para garantizar la escalabilidad y el mantenimiento del sistema, se ha seguido una estructura de directorios profesional, separando los datos, los modelos entrenados y las herramientas de apoyo.

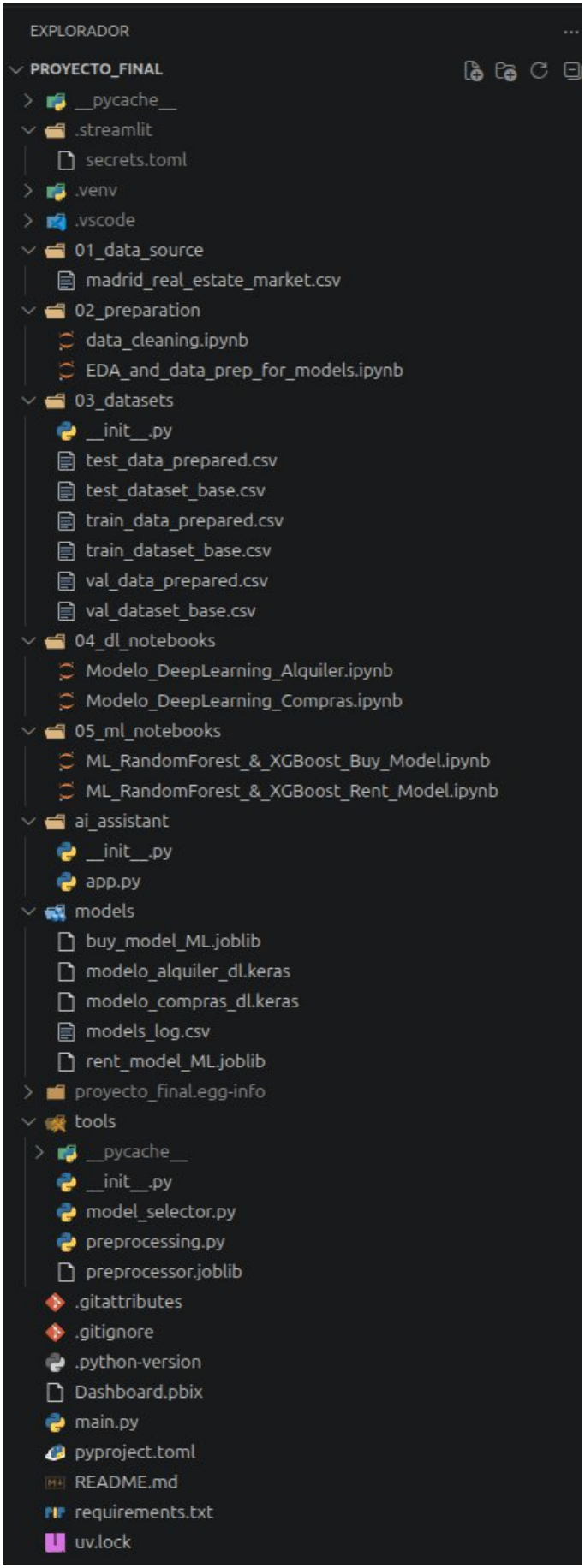


Figura 13. Organización del repositorio y estructura de carpetas del proyecto.

Adicionalmente, el repositorio incluye una carpeta llamada Documentación con la memoria técnica completa del proyecto, constituyendo el entregable final.

5.2. Control de Versiones y Desarrollo Iterativo

El desarrollo del software se ha gestionado mediante Git, permitiendo un historial de cambios detallado. Esto ha sido fundamental para experimentar con diferentes arquitecturas de redes neuronales y algoritmos de Machine Learning sin riesgo de pérdida de código estable.

Debido al gran tamaño que tienen los modelos de Machine Learning (aún comprimidos), desarrollados en Scikit Learn, se tuvo que hacer uso de Git LFS para poder subirlos a las actualizaciones del repositorio.

5.3. El Selector Inteligente de Modelos (Automatización)

Una de las innovaciones de la arquitectura es el archivo models_log.csv. En lugar de dejar el modelo fijo en la aplicación, se ha desarrollado una lógica que:

- Consulta el archivo models_log.csv.
- Identifica cuál es el modelo con el mejor R^2 registrado hasta la fecha.
- Carga automáticamente el archivo correspondiente, ya sea un formato .joblib (Machine Learning) o .keras (Deep Learning).
- El archivo obtiene los datos al momento de ejecutar cada notebook, este hace una verificación comparativa con respecto a los R^2 y si el modelo propuesto es mejor que el registrado para su tipo (Compra o Alquiler), se adiciona una línea al archivo y al momento de ejecutarse el asistente, éste buscará el mejor modelo en la lista e identificará el nombre del archivo correspondiente a ser invocado.

Esto permite que, si en el futuro se entrena un modelo mejor, la aplicación se actualice automáticamente sin necesidad de modificar una sola línea de código en el frontend.

En una próxima versión se intentará incorporar herramientas como **MLFlow**. Se realizó un primer intento, pero hubo problemas con algunas dependencias y requerían ajustes mayores en el resto del proyecto que requerirán más tiempo del disponible.

5.4. Interfaz de Usuario e Integración de LLM

La aplicación se ha desplegado utilizando Streamlit, ofreciendo una experiencia fluida en dos vertientes:

- Extracción de Datos con LLM (Groq)**:** Se utiliza la API de Groq (Llama 3.3) para analizar el texto libre de un anuncio inmobiliario pegado por el usuario y extraer automáticamente los campos técnicos necesarios (metros, habitaciones, barrio, extras). El LLM actúa como capa de interpretación entre el lenguaje humano y la entrada estructurada que requieren los modelos predictivos.

Adicionalmente, el mismo modelo gestiona el chat asesor de la pestaña principal, manteniendo el historial completo de la conversación mediante st.session_state para ofrecer respuestas contextuales en cada turno.

- Inferencia en Tiempo Real: Una vez extraídos los datos, los modelos de predicción calculan instantáneamente el precio justo de venta y alquiler.

5.4.1. Visualización Avanzada (Power BI)

Como complemento , se ha integrado un dashboard de Business Intelligence para la monitorización de métricas críticas del sistema. Este panel permite al administrador del sistema validar visualmente los KPIs principales del proyecto en una sola vista consolidada:

- Control de Precisión: Se muestran como indicadores principales el R^2 de ambos modelos (R^2 Venta: 0,899 y R^2 Alquiler: 0,826), junto con el Error Absoluto Medio de cada mercado (MAE Venta: 59.539 € y MAE Alquiler: 149,95 €/mes), correspondientes al registro de producción (model_log) del modelo desplegado.
- Comparativa de Modelos (Benchmarking): Se visualiza el rendimiento comparado de los dos enfoques evaluados — XGBoost y Deep Learning — permitiendo auditar de forma inmediata qué arquitectura ofrece mayor precisión y justificar la selección del modelo en producción.
- Tamaño del Training Set: El panel incluye una visualización del volumen de datos empleado en cada fase del entrenamiento, diferenciando el conjunto de entrenamiento del conjunto de test, lo que garantiza la trazabilidad y reproducibilidad del proceso.
- Pipeline de Trabajo: Se representa de forma visual el orden y la relevancia relativa de cada etapa del pipeline, desde la Ingesta de Datos hasta la Aplicación final, pasando por Limpieza, Feature Engineering, Selección de Modelo e Integración. Esto facilita la comprensión del flujo completo a perfiles no técnicos.

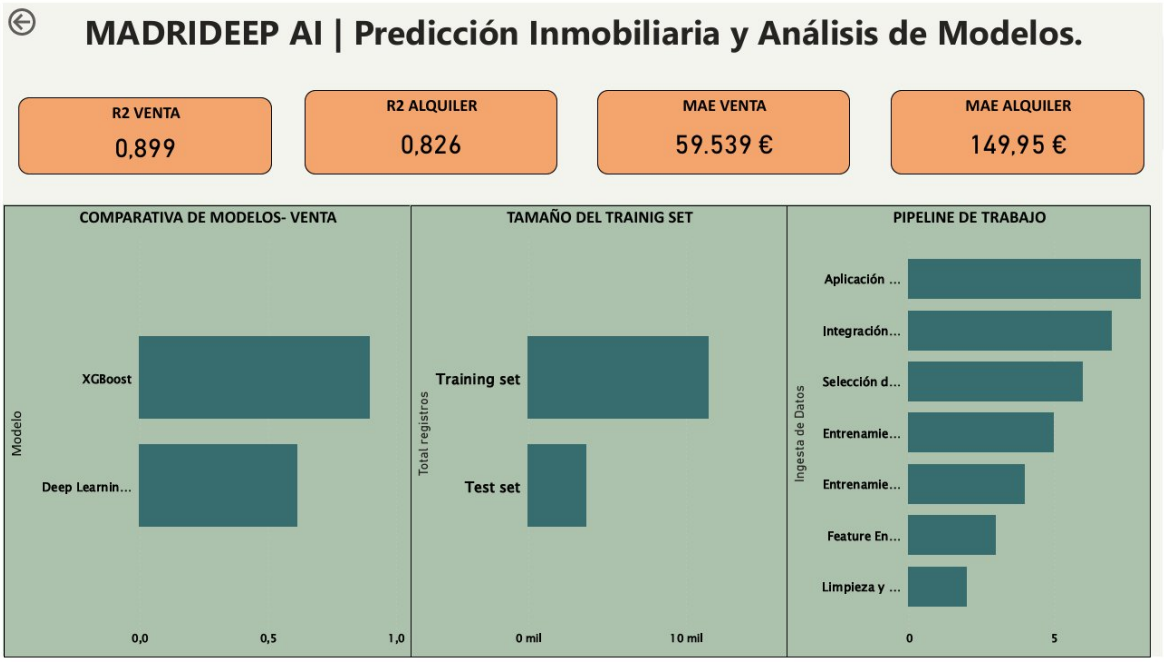


Figura 14. Dashboard de Business Intelligence (Power BI) para la monitorización del rendimiento de modelos, benchmarking de algoritmos y trazabilidad del pipeline de MadriDeep AI.

El sistema de visualización implementado se nutre directamente de las métricas registradas en el modelo en producción, permitiendo una auditoría continua del rendimiento sin necesidad de acceder al entorno de código. La utilidad de este panel se centra en dos pilares estratégicos:

- Democratización del Dato: Permite que perfiles no técnicos supervisen la fiabilidad de las tasaciones mediante una interfaz intuitiva, traduciendo los registros técnicos en información de negocio accionable.
- Gobernanza de Modelos: Facilita la comparación continua entre arquitecturas, asegurando que el modelo en producción sea siempre el de mayor rendimiento validado y manteniendo la trazabilidad completa del proyecto.

5.5. Entorno de Ejecución y Dependencias

Para asegurar que MadriDeep AI funcione en cualquier servidor, se ha generado un archivo requirements.txt que congela las versiones de las librerías críticas (scikit-learn, tensorflow-cpu, pandas). Esto garantiza la reproductibilidad, evitando que actualizaciones externas rompan el sistema.

Sin embargo, cabe destacar que en el proyecto se ha hecho uso de el gestor **uv** muy usado en proyectos modernos, de fácil uso y desempeño excepcional.

6. PRODUCTO FINAL

6.1. Demostración del Producto: MadriDeep AI

El resultado de este proyecto es una aplicación web plenamente funcional que integra inteligencia artificial generativa con modelos predictivos de alta precisión. La interfaz, desarrollada en Streamlit, permite una interacción dual:

- Asesor Virtual: Un chatbot que resuelve dudas específicas sobre el mercado madrileño.
- Tasador Inteligente: Una herramienta donde el usuario, simplemente pegando el texto de un anuncio, obtiene una valoración económica inmediata.

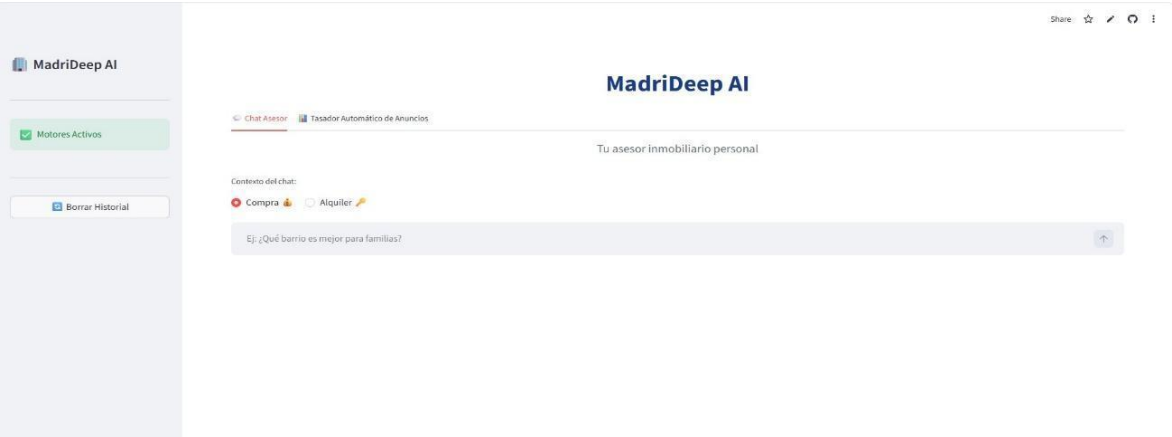


Figura 15. Interfaz final de la aplicación MadriDeep AI en funcionamiento.

7. IMPACTO DE NEGOCIO Y PROPUESTA DE VALOR

7.1. Problema de Negocio

El mercado inmobiliario en Madrid se caracteriza por una elevada volatilidad de precios y una fuerte asimetría de información. Tanto compradores como vendedores toman decisiones económicas relevantes basándose en referencias poco objetivas, lo que puede derivar en sobrevaloraciones o infravaloraciones significativas.

En este contexto, los principales problemas identificados son:

- Falta de transparencia en la fijación de precios.
- Dependencia de la experiencia subjetiva en procesos de tasación.
- Dificultad para analizar rápidamente oportunidades de inversión.
- Alto coste de error en operaciones inmobiliarias.

7.2. Propuesta de Valor

MadriDeep AI introduce un sistema de tasación automatizado basado en inteligencia artificial capaz de:

- Proporcionar valoraciones precisas en tiempo real.
- Reducir la incertidumbre en la toma de decisiones.
- Democratizar el acceso a herramientas avanzadas de análisis de mercado.
- Democratizar el acceso a herramientas avanzadas de análisis de mercado.
- Ofrecer asesoramiento personalizado mediante un asistente conversacional capaz de interpretar anuncios inmobiliarios y resolver consultas sobre el mercado madrileño.

7.3. Impacto Económico Potencial

El modelo arroja un Error Absoluto Medio (MAE) de 59.896 € en el mercado de compra. Tomando como referencia el precio medio de una vivienda en Madrid (en torno a los 300.000 €), esto representa un margen de error inferior al 2%, frente al error típico del sector que puede oscilar entre el 5% y el 10% en tasaciones tradicionales. En el mercado del alquiler, el MAE de 149,85 €/mes sobre un precio medio de 1.200 € supone igualmente una precisión superior al 87%.

Gracias a los modelos predictivos con un R^2 de 0,924 en compra y 0,826 en alquiler, MadriDeep AI ofrece una ventaja competitiva cuantificable para los distintos perfiles de usuario:

- Ahorro directo para compradores al evitar sobrepago.
- Mayor rentabilidad para inversores al identificar oportunidades.
- Optimización de precios para vendedores e inmobiliarias.

7.4. Modelo de Negocio

El proyecto presenta diversas vías de monetización:

- Modelo freemium para usuarios particulares, con acceso básico gratuito y funcionalidades avanzadas mediante suscripción.
- Licencias B2B dirigidas a agencias inmobiliarias, tasadoras y portales digitales.
- Integración mediante API para terceros interesados en incorporar el sistema de tasación.

Este enfoque híbrido permite escalar el producto tanto a nivel de usuario final como empresarial.

8. CONCLUSIONES

8.1. Conclusiones Técnicas

Tras semanas de experimentación y desarrollo, se han alcanzado las siguientes conclusiones:

- Dominio del Machine Learning: Los algoritmos basados en árboles (Random Forest) han demostrado ser superiores para este dataset tabular, logrando un R^2 de 0,924 en el modelo de compra. Esto significa que el modelo es capaz de explicar el 92% de la variabilidad del precio en Madrid.
- Aprendizaje del Deep Learning: Aunque las Redes Neuronales son potentes, la inestabilidad matemática al transformar logaritmos de precios altos supuso un reto técnico que se logró resolver mediante BatchNormalization, aunque finalmente se optó por ML Clásico para producción por su mayor eficiencia.

- LLM como Puente**:** La integración de Llama 3.3 (Groq) ha sido uno de los mayores aciertos del proyecto, actuando como extractor de entidades que permite a cualquier usuario sin conocimientos técnicos obtener una tasación simplemente pegando el texto de un anuncio. El LLM traduce lenguaje humano en datos estructurados de forma instantánea.

8.2. Consecución de Objetivos

Se han cumplido todos los objetivos planteados al inicio del proyecto:

- Extracción e integración de datos: Pipeline completo implementado fusionando tres fuentes heterogéneas (Kaggle, Datos.gob.es y Fotocasa), generando un dataset consolidado de más de 21.740 registros limpios y listos para el entrenamiento.
- Modelado predictivo de alta precisión: El modelo Random Forest alcanzó un R^2 de 0,924 en compra (MAE de 59.896 €) y un R^2 de 0,826 en alquiler (MAE de 149,85 €/mes), superando los objetivos de precisión establecidos al inicio del proyecto.
- Despliegue en producción: Aplicación web operativa en Streamlit con un sistema de MLOps personalizado (model_selector.py) que selecciona y carga automáticamente el mejor modelo registrado, integrando el LLM Llama 3.3 vía Groq como extractor de entidades para transformar texto libre de anuncios en datos estructurados.

8.3. Futuras Líneas de Mejora

Aunque el producto es funcional, el equipo visualiza varias vías para escalar MadriDeep AI:

- Visión Artificial: Integrar un modelo que analice las fotos del anuncio para ajustar el precio según el estado real de conservación de la vivienda.
- Expansión Geográfica: Ampliar el dataset para cubrir otras grandes capitales españolas.
- Módulo Financiero: Conexión con APIs bancarias para ofrecer el cálculo de la cuota hipotecaria según la tasación obtenida.